

# Final Project

Yunmin Go

School of AICE

**Handong Global University**

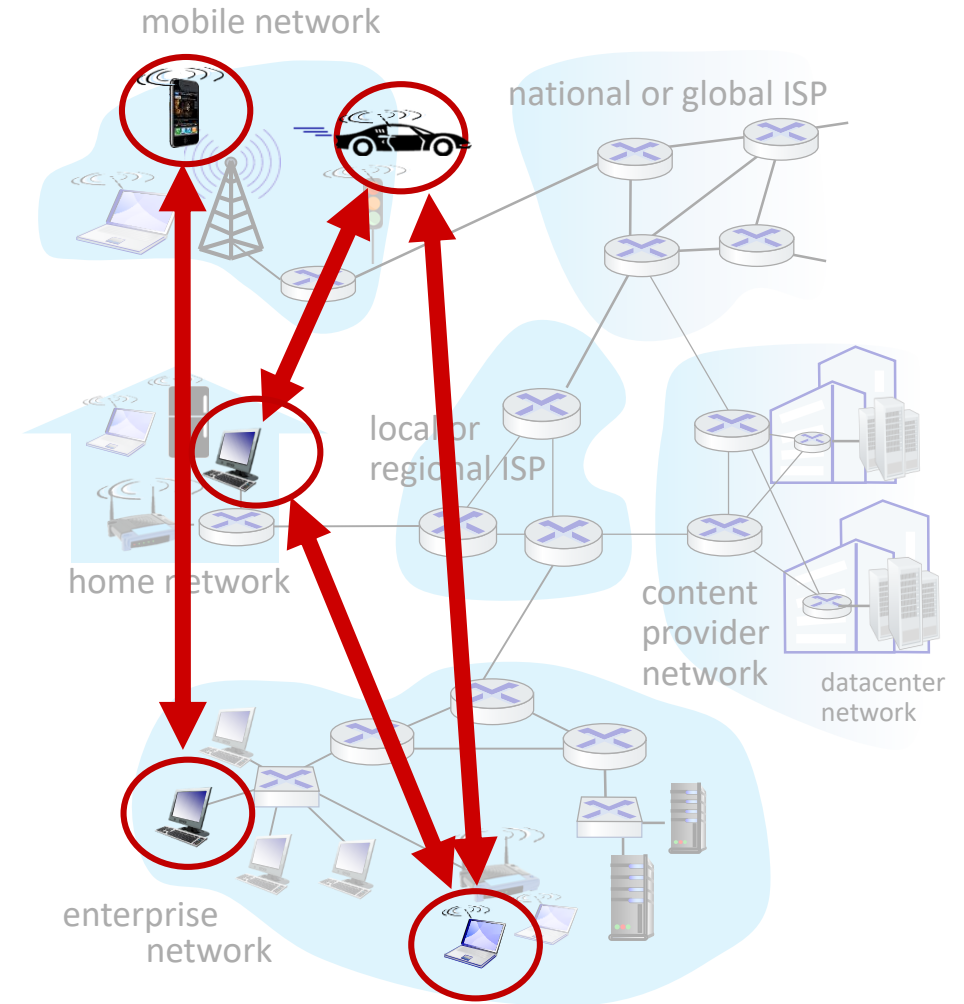
# Final Projects

- P2P for File Sharing
- Multi-Player Game: 판 뒤집기

# P2P for File Sharing

# Peer-to-peer (P2P) architecture

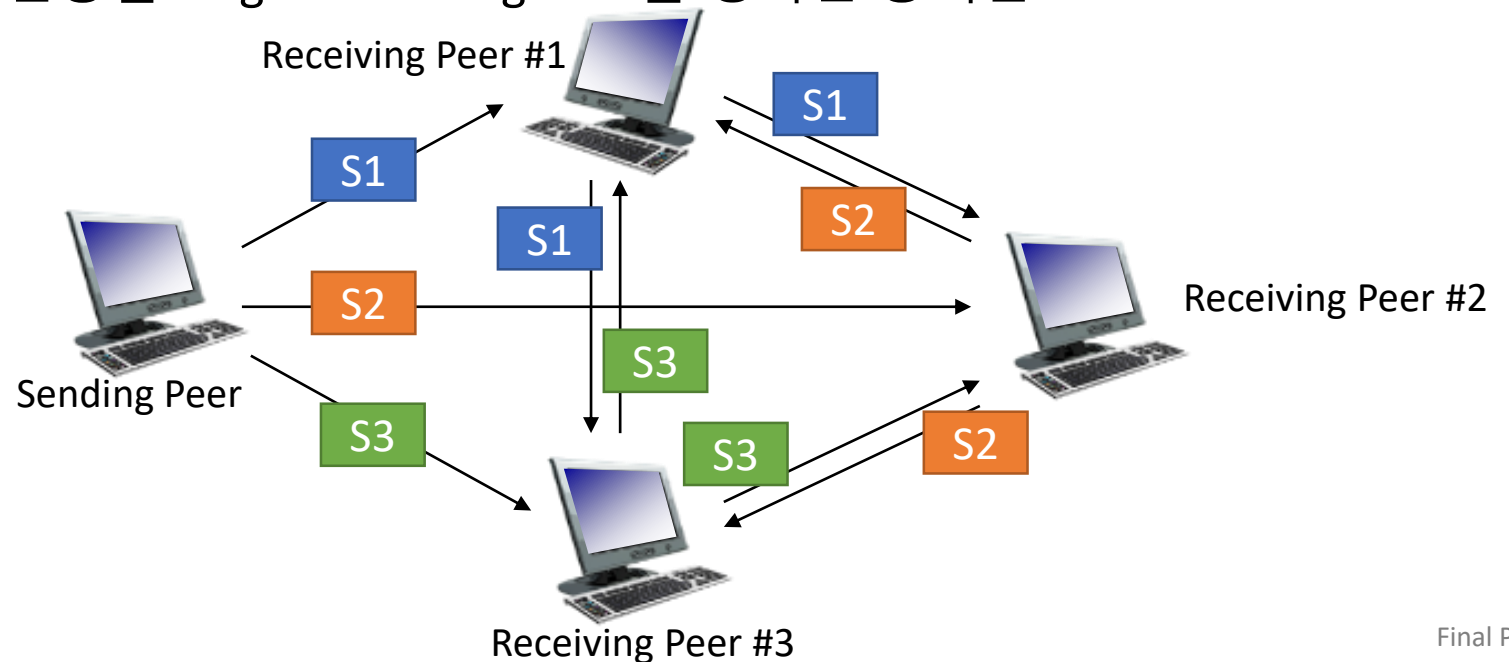
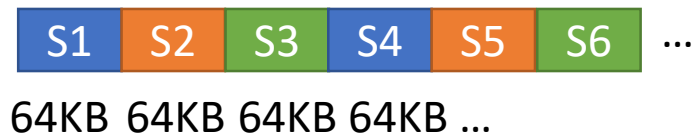
- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
  - *self scalability* – new peers bring new service capacity, and new service demands
- peers are intermittently connected and change IP addresses
  - complex management
- examples: P2P file sharing (BitTorrent), streaming (KanKan), VoIP (Skype), Blockchain



# P2P for File Sharing

- File 공유를 위한 P2P 프로그램을 구현하는 것이 목적입니다.
- Sending Peer가 가진 File을 나머지 N 개의 Receiving Peer들과 공유하는 프로그램입니다.
- Sending Peer가 가진 File은 여러 개의 Segment로 나눕니다. 이 때 Segment의 크기는 동일합니다.
- Sending Peer는 임의의 Segment를 단 하나의 Receiving Peer에게만 전송합니다.
- Sending Peer가 Segment를 전송할 Target Receiving Peer를 정하는 방식은 Round Robin 방식을 사용합니다.

File Size



# P2P for File Sharing

- 실행 파일의 이름은 p2p 이며, 아래와 같은 옵션을 지원할 수 있어야 합니다.

-s : sending peer로 실행  
-r : receiving peer로 실행  
-n : sending peer에서 받아들일 수 있는 최대 receiving peer 수  
-f : sending peer의 전송 File 이름  
-g : segment 크기, 단위는 KB  
-a : receiving peer일 경우 sending peer의 IP와 Port 정보 입력  
-p: listen port 번호

Example

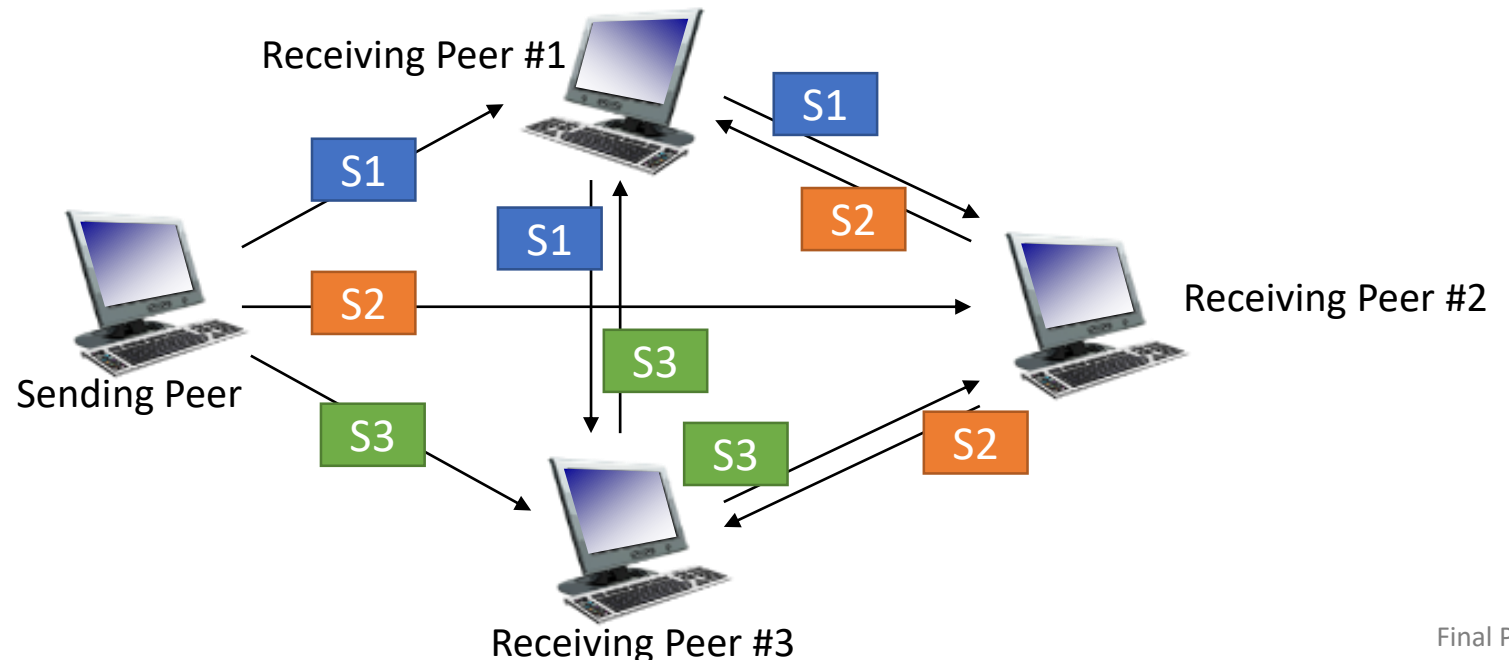
```
$ ./p2p -s -n 3 -f hgu.mp4 -g 64
```

```
$ ./p2p -r -a 192.168.10.2 9090
```

File Size



64KB 64KB 64KB 64KB ...



# Simple P2P for File Sharing

- 프로그램을 실행하면 파일 송수신 상태를 확인할 수 있는 Progress Bar를 출력하세요.

Example

```
$ ./p2p -p 9090 -s -n 3 -f hgu.mp4 -g 64
```

... (자유롭게 상태 메시지 출력 가능)

```
Sending Peer [#####] 40% (4194304/10485760Bytes) 12.8Mbps (2.5s)
```

```
To Receiving Peer #1 : 12.2Mbps (1441762 Bytes Sent / 0.9s)
```

```
To Receiving Peer #2 : 13.1Mbps (1376256 Bytes Sent / 0.8s)
```

```
To Receiving Peer #3 : 13.1Mbps (1376256 Bytes Sent / 0.8s)
```

```
$ ./p2p -p 9091 -r -a 192.168.10.2 9090
```

... (자유롭게 상태 메시지 출력 가능)

```
Receiving Peer 1 [#####] 40% (4194304/10485760Bytes) 12.8Mbps (2.5s)
```

```
From Sending Peer : 12.2Mbps (1441762 Bytes Sent / 0.9s)
```

```
From Receiving Peer #2 : 13.1Mbps (1376256 Bytes Sent / 0.8s)
```

```
From Receiving Peer #3 : 13.1Mbps (1376256 Bytes Sent / 0.8s)
```

※ Receiving peer는 수신에 대해서만 출력

- Makefile로 컴파일 할 수 있어야 합니다.

# Multi-Player Game



# Multi-Player Game:

- 판 뒤집기 게임: <https://www.youtube.com/watch?v=MaBvizl5qrc>



# Multi-Player Game

## ■ 판 뒤집기 게임의 규칙

- 이 게임에서는 짝수 명의 플레이어가 참여할 수 있습니다.
- 플레이어들은 반으로 나뉘어 두 팀이 게임을 진행합니다.
- 게임을 시작하면  $N \times N$  공간 임의의 위치에  $x$ 개(짝수)의 판이 위치합니다.
- 판은 양면이며 각 면은 서로 다른 색(예: 빨강/파랑)으로 표시해야 합니다.
- 각 팀에 색상이 배정되어 합니다. (예: A팀은 빨강, B팀은 파랑)
- 게임 초기에 배치된 판의 색상 수는 양 색상이 동일해야 합니다.
- 게임이 시작되면 플레이어는 판이 있는 곳으로 이동하여 판을 뒤집을 수 있습니다.
- 게임은  $\tau$  초 동안 진행합니다.
- $\tau$  초의 시간 이후 더 많은 색상을 보이게 한 팀이 승리합니다.

# Multi-Player Game

## ■ 구현 고려 사항

- 서버를 시작할 때 Command line argument를 통해 아래와 같은 것들을 지정할 수 있습니다.
  - -n: 참여하는 플레이어의 수 ('-n 2'는 플레이어 2명, '-n 8'은 플레이어 8명 참여)
  - -s: 판이 놓이는 2차원 공간의 크기 ('-s 100'은 100x100 공간, '-s 50'은 50x50 공간)
  - -b: 판의 개수 ('-b 20'은 판 20개)
  - -t: 게임 진행 시간 ('-t 60'은 60초, '-t 100'은 100초)
  - -p: 서버의 Listen 포트 번호
- 플레이어는 터미널을 통해 서버에 접속하여 게임을 진행합니다.
- 플레이어는 '화살표 키'를 조작하여 이동할 수 있습니다
- 플레이어는 '엔터키'를 눌러 판을 뒤집을 수 있습니다.
- 현재의 모든 게임 상태는 각 플레이어의 화면에 출력되어야 합니다.
  - 모든 플레이어의 위치, 판의 상태, 남은 시간 등...
- 상기 언급되지 않은 것들은 임의로 결정하여 진행하면 됩니다.
- Makefile로 컴파일 할 수 있어야 합니다.

# Due Date

- 팀 별 설계: 7/27 (월)
- 개인별 구현: 7/31 (금)